

REPORT

Pattern Recognition and Machine Learning

Lab01 - Classification with Linear Models

22KDL- Mathematics and Computer Science

November 3, 2024 - University of Science, VNU-HCM

I. INFORMATION ABOUT MEMBERS

NO	NAME	ID STUDENT	DISTRIBUTION (%)
1	Trần Đăng Khoa	20280054	
2	Phạm Trần Tấn Phát	20280070	
3	Trần Bình Phương	22280071	
4	Nguyễn Hồng Sơn	22280078	
5	Lê Tuấn Minh Thành	22280083	
6	Nguyễn Trường Thịnh	22280086	
7	Võ Minh Thịnh	22280087	
8	Từ Thức	22280093	

II. INTRODUCTION ABOUT DATASET

1. Abstract:

We present Fashion-MNIST, a new dataset consisting of 28x28 grayscale images of 70,000 fashion products from 10 categories, with 7,000 images per category. The training set has 60,000 images and the test set has 10,000 images. Fashion-MNIST is intended to serve as a direct drop in replacement for the original MNIST dataset for benchmarking machine learning algorithms, as it shares the same image size, data format and the structure of training and testing splits. The dataset is freely available at “<https://github.com/zalandoresearch/fashion-mnist>.”

2. Detail of Fashion-MNIST

The original MNIST dataset from 1998 is a popular 10 category dataset consisting of 70,000 examples of handwritten digits. It was first introduced by LeCun et al. MNIST is a modified version of handwritten data obtained from NIST, the National Institute of Standards and Technology. The black and white images from NIST were normalized into a 20x20 pixel box, which preserved their aspect ratio. This resulted in grayscale images due to the anti-aliasing technique used in normalizing the images. Finally, the images were centered in a 28x28 image based on the mass of the pixels .

Originally created in 1998, MNIST has become an extremely popular benchmark dataset for its ease of use in prototyping and testing new classifiers. It is particularly popular in deep learning due to its small size compared to other datasets . In 2012, a record high accuracy of 99.77% was achieved using deep neural networks .

Fashion-MNIST was introduced in 2017 by Xiao et al. in order to provide a similarly sized alternative to MNIST which poses a more challenging classification challenge. Fashion-MNIST retains the same data structure of grayscale images as MNIST. Hence, it is possible to use it as a prototyping dataset in the same scenarios as MNIST. Since classification accuracies have become so high for the MNIST dataset, Fashion-MNIST provides a more challenging alternative which can easily replace MNIST, allowing for better comparisons between modern deep learning models.

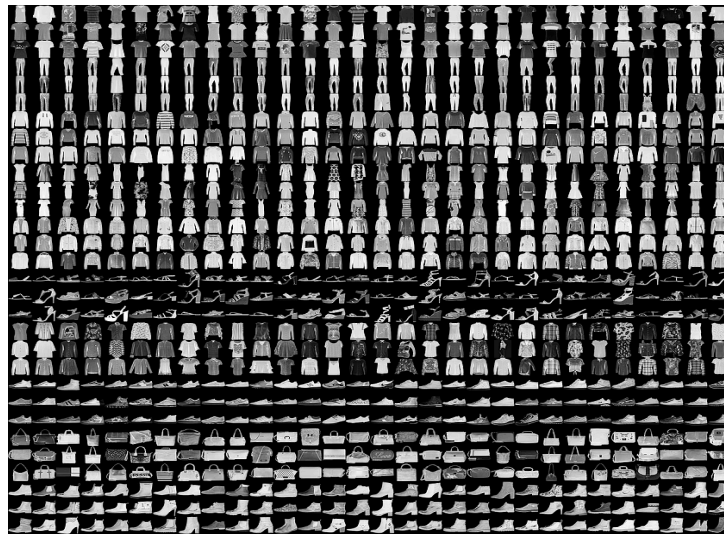


Fig. 1. Fashion-MNIST Dataset Images

3. Problem

In this assignment, our group will apply **Linear models** and **SVMs** to classify fashion items from the **Fashion-MNIST** dataset. We'll also explore how the curse of dimensionality effects model performance and use techniques like dimensionality reduction to mitigate it.

III. IMPLEMENTATION STEPS

A. DATA LOADING AND PREPROCESSING

- Load data from open source code (uploaded on github)
- Convert data from 'numpy' array to 'cupy' array to use on GPU
- Scale Data
- Exploratory Data Analysis:
 - Show some images of each class
 - Check the distribution of classes in the train_images set
 - Visualize the data set using UMAP in 2D space
 - Determine the pixel distribution of each class
 - Check the pixel correlation of each pair of classes

B. BUILT MODELS

In this section we use cuML instead of scikit-learn to enable GPU-accelerated training. As part of NVIDIA's RAPIDS AI suite, cuML speeds up model training on large datasets by leveraging GPU power, which scikit-learn doesn't support. And the processing unit is GPU T4 x2 on Kaggle.

We divide the model building into two parts: the first part is Linear Model - Logistic Regression, the remaining part is Support Vector Machine (SVM). In each part we will use different methods to build models and train them. In this section, we also focus on comparing each model independently within two scenarios: before dimensionality reduction and after dimensionality reduction as well as between Logistic Regression and Support Vector Machine Models. Specifically implemented in the notebook file, the following is a summary.

1. Linear Model - Logistic Regression

In the section on building linear models, we explored the mechanics of **Logistic Regression** and **Softmax Regression**. We found that, given the Fashion-MNIST dataset, which contains multiple classes with relatively high correlation, logistic regression faced challenges due to its classification approach, making some classes difficult to distinguish accurately. Consequently, we faced several considerations and challenges when selecting and constructing our model. Through research and analysis, we decided to implement and test both the **One-vs-One (OvO)** and **One-vs-Rest (OvR)** methods.

Softmax in statistics is a classification method that generalizes logistic regression to multiclass problems, i.e. with more than two possible discrete outcomes. That is, it is a model that is used to predict the probabilities of the different possible outcomes of a categorically distributed dependent variable, given a set of independent variables (which may be real-valued, binary-valued, categorical-valued, etc.)

One-Vs-Rest: Here, we create classifiers between one class and all other classes, then select the class with the highest probability as the predicted class for the data point.

One-Vs-One: In this approach, for each pair of different classes, we create a binary classifier to distinguish between them, then use voting to determine the winning class. Finally, we choose the class with the most votes as the predicted class for the data point.

Below are statistics on training time and model performance of each method:

Model	Softmax	OvR	OvO
Training time (s)	76.97	21.75	16.10
Accuracy	0.844	0.841	0.850
Precision	0.843	0.839	0.849
Recall	0.844	0.841	0.850
F1 Score	0.843	0.840	0.849

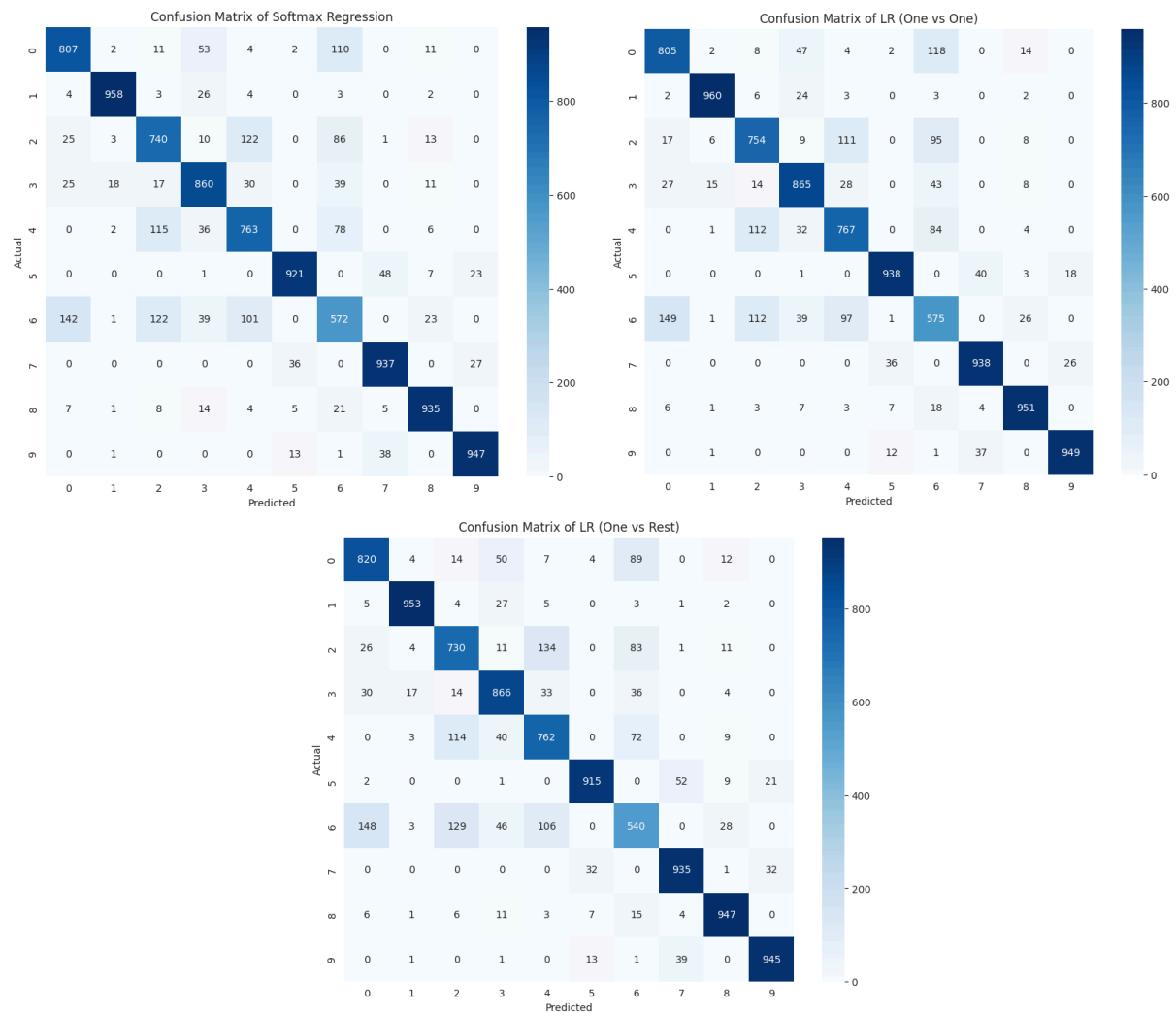
Softmax has the longest training time (3.5 times OvR and 4.8 times OvO), but its performance results are not significantly better than **OvO**.

OVR, while having faster training time than **Softmax**, is still less effective than **OVO** in terms of performance metrics. And even the training time is higher than OvO although it is not significant compared to the difference with Softmax

OvO is clearly the most effective model in terms of both training time and performance metrics.

Based on these results, OVO seems to be the best choice for your classification task, as it optimizes both model training time and performance.

Below are statistics confusion matrix:



Softmax: Class 'Trouser', 'Sandal', 'Sneaker', 'Bag', 'Ankle' have high accuracy with over 900 correct predictions and minimal misclassifications. There are 4 pairs of classes that are easily confused: 'T-shirt/top' vs 'Shirt', 'Pullover' vs 'Coat', 'Shirt' vs 'Pullover', 'Shirt' vs 'Coat'.

OvR: Class 'Trouser', 'Sandal', 'Sneaker', 'Bag', 'Ankle' have high accuracy with over 900 correct predictions and minimal misclassifications. (almost similar to the softmax model). There are 4 pairs of classes that are easily confused: 'T-shirt/top' vs 'Shirt', 'Pullover' vs 'Coat', 'Shirt' vs 'Pullover', 'Shirt' vs 'Coat'.

OvO: Class 'Trouser', 'Sandal', 'Sneaker', 'Bag', 'Ankle' have high accuracy with over 900 correct predictions and minimal misclassifications. (similar to the softmax regression model). There are

4 pairs of classes that are easily confused: 'T-shirt/top' vs 'Shirt', 'Pullover' vs 'Coat', 'Shirt' vs 'Pullover', Confusion between 'Shirt' and 'Coat' is reduced compared to the previous model (96)

2. Support Vector Machine (SVM)

In building linear models, we chose to explore Support Vector Machines (SVMs) as a robust alternative due to their effectiveness in handling high-dimensional data. SVMs work by finding the optimal hyperplane that maximally separates classes in the feature space. They are particularly beneficial for datasets like Fashion-MNIST, where complex patterns and overlapping classes exist, due to their flexibility in choosing different kernels. We decided to choose two types of kernels for model building and evaluation: the **Linear Kernel** and the **RBF Kernel**.

Linear Kernel: The linear kernel is straightforward and works effectively for linearly separable data, creating a single hyperplane to separate classes. It has low computational cost and is effective when class boundaries are approximately linear.

RBF Kernel: For more complex class distributions, we used the Radial Basis Function (RBF) kernel, which maps data into a higher-dimensional space, allowing non-linear decision boundaries. The RBF kernel adjusts to local variations in data, making it suitable for classes that overlap or have non-linear separations.

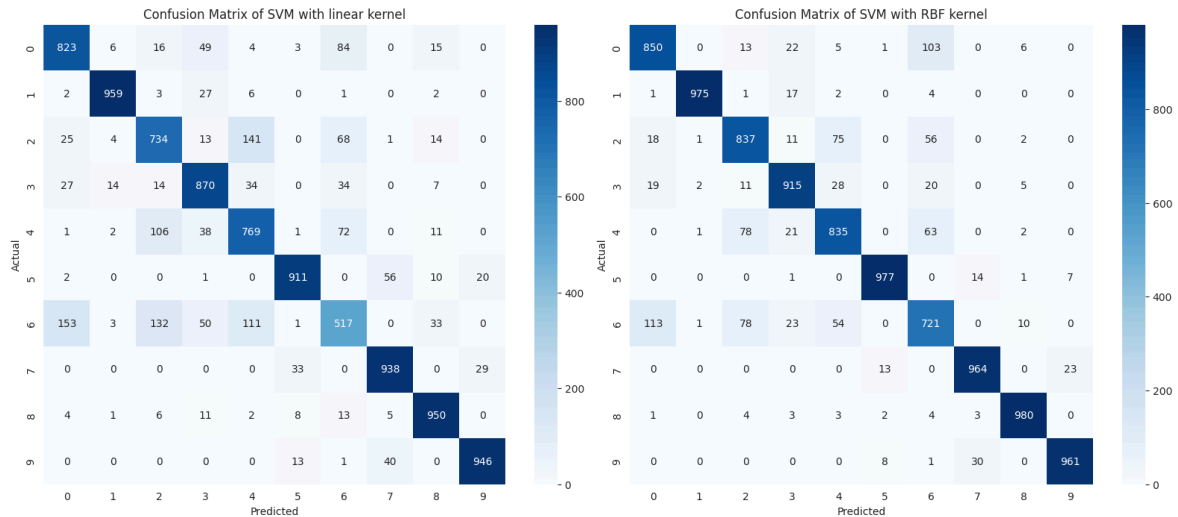
Below are statistics on training time and model performance of each method:

Model	Linear	RBF
Training time (s)	4.04	23.03
Accuracy	0.842	0.901
Precision	0.840	0.901
Recall	0.842	0.902
F1 Score	0.840	0.901

Training time: The **Linear Kernel** has a much faster training time compared to the **RBF kernel**. This may be because the **Linear Kernel** only needs to compute a linear separating boundary between classes, whereas the **RBF Kernel** performs nonlinear computations to separate more complex data classes.

Metrics: The metrics for the **RBF kernel** are all higher than those for the linear kernel, indicating that the **RBF kernel** has better classification performance and is more effective at handling complex patterns. We can see that the **RBF kernel** performs better across all metrics (accuracy, precision, recall, F1 score), making it suitable for applications requiring higher accuracy.

Below are statistics confusion matrix:



Linear Kernel: Class 'Trouser', 'Sandal', 'Sneaker', 'Bag', 'Ankle' have high accuracy with over 900 correct predictions and minimal misclassifications. There are 4 pairs of classes that are easily confused: 'Pullover' vs 'Coat', 'Shirt' vs 'Coat', 'Shirt' vs 'Pullover', 'Shirt' vs 'T-shirt/top'

RBF Kernel: Class 'Trouser', 'Dress', 'Sandal', 'Sneaker', 'Bag', 'Ankle' have high accuracy with over 900 correct predictions and minimal misclassifications. Notably, the inclusion of 'Dress' shows improved performance compared to other models, indicating that the model is effectively distinguishing this class as well. There is only 1 pair of classes that are easily confused: 'T-shirt/top' vs 'Shirt'.

3. Comparison between Logistic Regression and SVM models (Full Datasets)

In our evaluation, the SVM with the RBF kernel outperformed logistic regression on the Fashion-MNIST dataset due to its ability to handle the nonlinear, complex relationships present in the data. Fashion-MNIST consists of grayscale images across ten classes, many of which contain visual similarities that pose challenges to clear separation. For example, items like “T-shirt/top” and “Shirt” often exhibit overlap due to similar shapes and textures, making it difficult for linear models like logistic regression to accurately distinguish these classes. Logistic regression models, including OvO and OvR, are effective for linearly separable classes, but with Fashion-MNIST’s correlated classes, particularly categories like "T-shirt/top" and "Shirt," logistic regression struggled to differentiate, as it relies on linear decision boundaries.

In contrast, the SVM with an RBF kernel effectively handles these nonlinearities by mapping data to a higher-dimensional space, where it finds optimal separation boundaries between closely related classes, leading to a clear boost in accuracy, achieving around 90% compared to logistic regression’s plateau at 84-85%. This adaptability of the RBF kernel to complex data structures and its flexibility in accommodating varying shapes in high-dimensional space make it well-suited to Fashion-MNIST’s multi-class structure.

While the RBF SVM has a longer runtime than linear kernel or logistic regression models, this trade-off is justified by the performance gains. Linear SVM models and logistic regression, though efficient in runtime, showed limitations in capturing Fashion-MNIST’s intricacies, whereas the RBF kernel SVM’s margin-maximization property and regularization help reduce misclassification in this complex, correlated dataset. Given Fashion-MNIST’s unique challenges, including visually similar and overlapping classes, the RBF kernel SVM is the preferred choice due to its capacity to effectively capture Fashion-MNIST’s subtle, nonlinear class distinctions.

4. Models after Reduction

Based on our analysis of the explained variance in principal components, we plan to retrain our models using two different configurations: one with **n_components = 25** and another with **n_components = 100**. This approach allows us to evaluate how varying the dimensionality of the data impacts model performance and interpretability.

With n_components = 100:

Model	Softmax	OvR	OvO	Linear	RBF
Training time (s)	6.974	2.047	3.233	0.355	9.365
Accuracy	0.841	0.838	0.847	0.834	0.900
Precision	0.839	0.835	0.846	0.831	0.900
Recall	0.841	0.838	0.847	0.834	0.900
F1 Score	0.840	0.836	0.846	0.831	0.900

Despite the reduction in dimensions to 100 components using PCA, there is almost no change in model performance. This suggests that PCA effectively captures the essential features in the data, reducing dimensionality without significant loss in predictive quality. Performance metrics are clustered around 0.83 to 0.90, indicating that all models maintain strong accuracy even after dimensionality reduction.

With n_components = 25:

Model	Softmax	OvR	OvO	Linear	RBF
Training time (s)	2.572	0.539	1.786	0.139	6.478
Accuracy	0.809	0.798	0.821	0.709	0.879
Precision	0.807	0.792	0.819	0.783	0.879
Recall	0.809	0.798	0.821	0.790	0.879
F1 Score	0.807	0.794	0.820	0.782	0.879

Metrics: After reducing to 25 components, there is a slight decrease in performance metrics, which is expected given the reduction in dimensions. However, all models still maintain relatively high scores, with accuracy, precision, recall, and F1 scores ranging from approximately 0.79 to 0.88. This indicates that PCA with 25 components captures essential information, though some details are lost compared to 100 components.

Training Time: Significantly reduced compared to 100 components on all models. Regarding speed comparison between models, it is still the same as above. Linear is still very fast and RBF is many times slower.

In summary, SVM Linear and OvR are optimal for computational efficiency, especially as dimensionality decreases, making them ideal for rapid applications.

SVM RBF and OvO offer higher accuracy at the cost of training time, suited for applications where accuracy outweighs computational cost.

Softmax provides a balanced solution, excelling in interpretability, making it suitable for probabilistic multiclass problems.

5. Retrain model with HOG feature extraction (Expansion)

C. ROLE OF DIMENSIONALITY REDUCTION IN MACHINE LEARNING

Dimensionality reduction, as demonstrated here through Principal Component Analysis (PCA), is an essential approach to mitigating the curse of dimensionality, a challenge where adding more features makes data increasingly sparse and difficult for models to generalize from. High-dimensional data often carries redundant or irrelevant information, leading to longer training times, overfitting, and difficulty in identifying relevant patterns. PCA addresses this by compressing the feature space into the most significant components, making models more efficient and generalizable.

In this evaluation, PCA with 100 components effectively reduced the dimensional space while preserving model accuracy. This reflects PCA's ability to capture core structures and variances within the data, reducing noise from less meaningful dimensions. By lowering dimensionality to 100, the models still maintained strong performance, suggesting that these components contain the critical information needed for accurate classification, even in a dataset as visually complex as Fashion-MNIST. This compression of essential information into fewer dimensions combats the curse of dimensionality by making the data less sparse, thus allowing models to learn patterns more effectively.

Reducing dimensions further to 25 components introduced a small trade-off in accuracy, as expected, yet performance remained relatively high. This demonstrates that even at 25 components, PCA successfully distills essential data characteristics while reducing training times. By simplifying the feature space, PCA minimizes overfitting, enhances generalization, and makes training times more feasible for computationally demanding algorithms, such as SVM with an RBF kernel. The time saved with fewer dimensions also highlights how dimensionality reduction makes complex modeling more practical for high-dimensional datasets.

In summary, PCA mitigates the curse of dimensionality by:

- + Reducing data sparsity, making it easier for models to find patterns.
- + Enhancing computational efficiency, allowing for faster model training.
- + Improving generalization and robustness by focusing on core, informative features.

By making high-dimensional data more manageable, dimensionality reduction with PCA helps models maintain high accuracy and efficiency, making it a powerful technique for handling the challenges posed by the curse of dimensionality.